

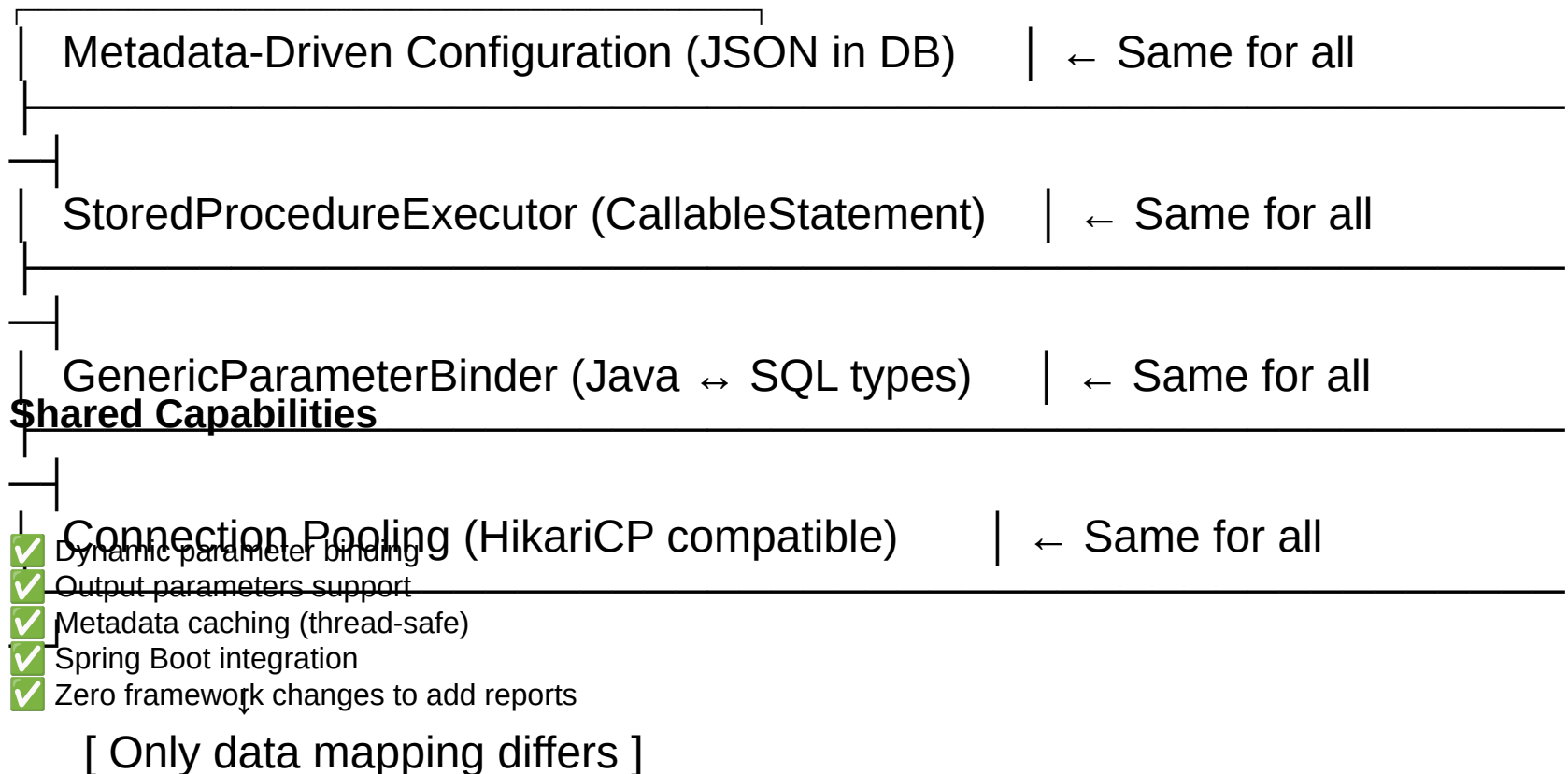
Executive Summary

****Business Problem:**** Metadata-driven reporting framework for SQL Server stored procedures

****Core Requirement:**** Add new reports without modifying

Common Foundation - What Doesn't Change

****All 3 approaches share the same core:****



****What differs:**** How ResultSet data is mapped and transformed

Option 1 - POJO Approach (Main Branch)

How It Works

// 1. Define POJO (compile-time type safety)

```
public class EmployeeReport {  
    private Integer employeeId;  
    private String firstName;  
    private BigDecimal salary;  
    // getters/setters  
}
```

// 2. Metadata with explicit mappings

```
{  
    "columnMappings": [  
        { "column": "employee_id", "field": "employeeId", "required": true },  
        { "column": "first_name", "field": "firstName", "required": true }  
    ]  
}
```

Code per report | 40-60 lines (POJO + mappings)

Runtime dependencies | 3 (Jackson, SLF4J, JDBC)

Memory overhead | ~200 MB (base JVM)

// 3. Execute with full type safety

Startup time | < 1 second

ReportResult<EmployeeReport> result = service.execute("report", params);

API learning curve |  Zero (standard Java) :D

Option 2 - Dual Mode (Dataset-impl Branch)

How It Works

```
// NO POJO needed for dynamic mode!
```

```
// Execute dynamically
```

```
DataSet employees = service.executeDynamic("employees", params);
```

```
// Transform with custom fluent DSL
```

```
DataSet result = DataQuery.from(employees)
```

```
.filter(row -> row.getBigDecimal("salary").compareTo(valueOf(80000)) > 0)
```

```
.select("name", "salary", "hire_date")
```

```
.orderBy("salary").desc()
```

```
.limit(10)
```

```
.execute();
```

Metric | Value

```
// Type-safe accessors (no casting)-----
```

```
Code per report | 10-15 lines (dynamic mode)
```

```
for (DataRow row : result.getRows()) {  
    Runtime dependencies | 3 (Jackson, SLF4J, JDBC)
```

```
String name = row.getString("name"); // Type-safe
```

```
Memory overhead | ~220 MB (base JVM + DataSet)  
BigDecimal salary = row.getBigDecimal("salary"); // No casting
```

```
Startup time | <1 second
```

```
LocalDate hireDate = row.getLocalDate("hire_date");
```

```
API learning curve | 🚩 Medium (4 new classes)
```

Option 3 - Spark SQL (Spark-impl Branch)

How It Works

```
// Setup Spark (local JVM only, no cluster)
```

```
SparkConfig config = SparkConfig.builder()
```

```
.master("local[*]") // Local execution in Spring Boot JVM
```

```
.build();
```

```
// NO POJO, NO column mappings!
```

```
SparkReportResult result = service.execute("employees", params);
```

```
// Transform using Spark SQL (industry-standard API)
```

```
Dataset<Row> employees = result.getDataset();
```

```
Dataset<Row> summary = employees
```

```
    .filter("salary > 80000")
```

```
    .groupBy("department_id")
```

```
    .agg(
```

```
        avg("salary").as("avg_salary"),
```

```
        max("salary").as("max_salary"),
```

```
        count("*").as("employee_count")
```

```
    );
```

Development Effort Comparison

Code Required for Same Report

****Scenario:**** Employee report filtered by salary, grouped by department

****Code Reduction:**** POJO (55 lines) → Dataset-impl (12 lines, ****78% less****) → Spark-SQL (7 lines, ****87% less****)

Transformation Capabilities Comparison

Operation | POJO (Java Streams) | Dataset-impl (Custom DSL) | Spark-SQL

Filter |  `stream().filter()` |  `.filter(row -> ...)` |  `.filter("col > val")`

Map/Select |  `stream().map()` |  `.select("col1", "col2")` | 

`.select("col1", "col2")`

Sort |  `sorted(Comparator)` |  `.orderBy("col").desc()` | 

`.orderBy(desc("col"))`

Limit |  `limit(n)` |  `.limit(n)` |  `.limit(n)`

Summary GroupBy + Agg |  Manual `groupingBy()` + custom reducers | 

`.groupBy().avg().count()` |  `.groupBy().agg(avg(), count())`

Joins |  Manual Map-based joins |  `DataOperations.join()` | 

POJO: Native Java Streams excellent for simple operations, verbose for complex ones
Dataset-impl: DSL covers common operations, requires framework maintenance

Spark-SQL: Industry-standard API, richest feature set, well-documented

Window Functions |  Complex manual code |  Not supported |  Built-in

`.Window.partitionBy()`

Pivot |  Very complex |  `DataOperations.pivot()` |  `.pivot("column")`

Running Totals |  Manual stateful logic | 

`.DataOperations.withRunningTotal()` |  Window functions

Resource & Dependency Analysis

JAR Dependencies (Production)

Dependency	POJO	Dataset-impl	Spark-SQL
------------	------	--------------	-----------

jackson-databind	1.5 MB	1.5 MB	1.5 MB
------------------	--------	--------	--------

jackson-datatype-jsr310	0.1 MB	0.1 MB	0.1 MB
-------------------------	--------	--------	--------

slf4j-api	0.1 MB	0.1 MB	0.1 MB
-----------	--------	--------	--------

mssql-jdbc	1.2 MB	1.2 MB	1.2 MB
------------	--------	--------	--------

logback-classic	0.5 MB	0.5 MB	0.5 MB
-----------------	--------	--------	--------

Runtime Memory (Spring Boot on Tomcat)

Component	POJO	Dataset-impl	Spark-SQL
-----------	------	--------------	-----------

spark-sql (local)	-	-	**17 MB**
-------------------	---	---	-----------

Total JAR size	**~3.6 MB**	**~3.8 MB**	**~20.7 MB**
--------------------	-------------	-------------	--------------

Base JVM (heap)	256 MB	256 MB	256 MB
-----------------	--------	--------	--------

Spring Boot	150 MB	150 MB	150 MB
-------------	--------	--------	--------

Framework overhead	10 MB	20 MB	30 MB
--------------------	-------	-------	-------

SparkSession (local)	-	-	**+150 MB**
----------------------	---	---	-------------

Total recommended	**512 MB**	**512 MB**	**768 MB**
-----------------------	------------	------------	------------

Startup Time (Spring Boot Application)

Metric	POJO	Dataset-impl	Spark-SQL
--------	------	--------------	-----------

Framework init	<100 ms	<100 ms	<100 ms
----------------	---------	---------	---------

SparkSession init	-	-	**2-3 seconds**
-------------------	---	---	-----------------

First report execution	~200 ms	~200 ms	~500 ms
------------------------	---------	---------	---------

Decision Matrix

Scenario-Based Recommendations

Scenario | Recommended | Rationale

Stable schema, small team	✓ **POJO**	Compile-time safety, zero learning curve, native Java Streams
Changing schema frequently	✓ **Spark-SQL**	No POJOs to update, auto-inferred schema, 87% less code
Complex analytics (pivots, windows)	✓ **Spark-SQL**	Rich SQL operations, query optimization, industry standard API
Risk Assessment Risk POJO Dataset-impl Spark-SQL		
Minimal dependencies critical	✓ **POJO**	Only 3.6 MB total, no additional libraries
Code maintenance	⚠ High (many POJOs)	✓ Low (no POJOs) ✓ Very Low memory environment
Low memory environment	✓ **POJO** or **Dataset-impl**	512 MB vs 768 MB for Spark-SQL
Runtime errors	✓ Low (compile-time)	⚠ Medium (string column names)
REST API, JSON output	✓ **Spark-SQL**	⚠ Direct `.toJSON()` conversion
⚠ Medium (string column names)	✓ **POJO**	⚠ Native Java, or ⚠ **Dataset-impl**
Team unfamiliar with Spark	✓ None (Java stub)	⚠ High (56 custom methods) ✓ if willing to learn custom API
API maintenance	✓ None (Spark is maintained)	
Long-term maintenance concern	⚠ **POJO** or **Spark-SQL**	Standard APIs vs custom framework maintenance
Team onboarding	✓ Fast (<1 day)	⚠ Medium (2-3 days) ⚠ Medium (2-3 days with Spark training)
Dependency updates	✓ Simple (Jackson)	✓ Simple (Jackson) ⚠ Monitor Spark releases

Recommendation & Next Steps

Data-Driven Summary

Metric | POJO | Dataset-impl | Spark-SQL

Code per report | 55 lines | 12 lines (-78%) | 7 lines (-87%)

JAR size | 3.6 MB | 3.8 MB | 20.7 MB

Memory | 512 MB | 512 MB | 768 MB

Startup | <1s | <1s | 2-3s

API maintenance |  Zero |  56 methods |  Zero

Compile-Time Safety Type safety |  Compile-time (POJO with Java Streams) |  Runtime (Spark-SQL, -87% code) |  Runtime

Transformations |  Java Streams |  Custom DSL |  Spark SQL

Learning curve |  None |  Custom API |  Spark API

(Middle ground, custom API)

Choose POJO if

Compile-time type safety is ****critical****

Team wants native Java Streams (no new API)

Minimal dependencies/memory required

Willing to accept POJO maintenance overhead